

Highlights

End-to-End Ensemble Learning Architecture for Intrusion Attack Classification

- An end-to-end ensemble architecture is proposed for network intrusion classification.
- Spatial traffic relations are modeled by GAT embeddings combined with XGBoost.
- Temporal attack patterns are captured by a Residual CNN–BiLSTM learner.
- A meta-learner fuses complementary spatial and temporal predictions.
- Experiments on IoT-IDS-Zwave-2025 and CIC IIoT 2025 show robust performance.

End-to-End Ensemble Learning Architecture for Intrusion Attack Classification

*^aSchool of Information and Communications Technology, Hanoi University of Science
and Technology, Hanoi, Vietnam*

Abstract

Network-based Intrusion Detection Systems (NIDSs) must accurately identify increasingly sophisticated attacks from large-scale, imbalanced, and temporally evolving network flows. Our work proposes an end-to-end ensemble learning architecture for multi-class intrusion attack classification in IoT and IIoT environments. The proposed framework combines two complementary base learners. The first branch constructs a causality-preserving flow-based graph and applies Graph Attention Network embeddings [1] with XGBoost [2] to capture spatial relationships among network flows. The second branch transforms chronological traffic into sliding windows and uses residual convolutional blocks, Squeeze-and-Excitation recalibration, BiLSTM layers, and attention to model temporal attack behavior. A meta-learner then fuses the predictive distributions of both branches to obtain the final classification output. Experiments are conducted on three use cases from IoT-IDS-Zwave-2025 [3] and CIC IIoT 2025 [4], covering HTTP-based DDoS attacks, application-based DDoS attacks, and industrial IoT attack traffic. The results show that the proposed ensemble is competitive with traditional machine-learning, graph-based, and recent ensemble IDS baselines, achieving strong macro-F1 performance and improved robustness in challenging network traffic scenarios. In particular, the meta-learner benefits from the complementarity between spatial graph representations and temporal sequence representations, making the architecture suitable for heterogeneous intrusion detection settings.

Keywords: Intrusion Detection System, Network Security, Ensemble Learning, Graph Attention Network, XGBoost, Residual CNN-BiLSTM, IoT security

1. Introduction

The rapid expansion of information technology infrastructure has made the design of effective Intrusion Detection Systems (IDSs) an increasingly important research problem [5, 6]. Recent cybersecurity reports indicate that attacks are not only frequent but also increasingly sophisticated, which shifts the IDS objective from coarse anomaly detection toward accurate multi-class attack identification [7]. This requirement is particularly important for network-based IDSs (NIDSs), where defenders must classify high-volume network flows into benign traffic or specific attack categories in a timely and reliable manner.

Deep learning has become a promising direction for NIDS because it can learn nonlinear and high-dimensional representations from network traffic [8–10]. Existing architectures include Transformer-based traffic classifiers [11], CNN–BiLSTM models for sequential traffic behavior [12], and graph neural networks, including graph convolution and graph attention models [1, 13], that exploit communication topology [14–17]. In parallel, ensemble learning has shown strong empirical performance by combining complementary learners and reducing prediction variance [18, 19]. However, many IDS models still emphasize either spatial relations or temporal dynamics, while real attack traffic often contains both.

This paper proposes an end-to-end ensemble learning architecture for intrusion attack classification. The framework combines two base learners with complementary inductive biases. The first learner constructs flow-based graphs and uses Graph Attention Network (GAT) embeddings [1] with XGBoost [2] to capture spatial relations between network entities. The second learner uses sliding windows, residual convolutional blocks, and a BiLSTM attention module to model temporal dependencies. A meta-learner then fuses the outputs of both learners to obtain the final attack classification.

The main contributions are summarized as follows:

- Design a causality-preserving flow-as-node graph construction strategy and a GAT–XGBoost spatial learner that captures relationships among temporally related flows while avoiding temporal leakage. The graph-enhanced learner achieves the strongest Macro F1 and AUC-ROC in the HTTP-based DDoS use case.
- Develop a Residual CNN–BiLSTM temporal learner with sliding-window sequence modeling, residual feature extraction, and attention-based ag-

gregation. This learner consistently outperforms its ablated variants and achieves the best Macro F1 and AUC-ROC in the more challenging CIC IIoT 2025 use case.

- Propose a stacked ensemble framework that fuses the predictive distributions of the spatial and temporal learners through logistic-regression and XGBoost meta-learners. The ensemble improves robustness by exploiting complementary spatial and temporal evidence, especially when attack classes are imbalanced or harder to separate.
- Provide a comparative evaluation on three use cases from IoT-IDS-Zwave-2025 and CIC IIoT 2025 against traditional machine-learning models, graph-based IDS architectures, and recent ensemble IDS baselines. The results show that the proposed ensemble achieves the best Macro F1 in Use Cases 1 and 3 and remains competitive in Use Case 2.

2. Related Work

Network intrusion detection has evolved from rule-based and traditional machine-learning methods toward deep-learning-based models that can adapt to complex traffic patterns. Conventional classifiers such as decision trees, support vector machines, logistic regression, and random forests remain useful baselines, but their ability to model high-dimensional, imbalanced, and temporally evolving attack traffic is limited.

Sequential deep learning models are widely used to detect temporal dependencies in packet or flow sequences. CNNs extract local feature patterns, while recurrent models such as LSTM and BiLSTM capture long-range dependencies [8, 9, 12]. Hybrid CNN-BiLSTM architectures have therefore become common in IoT intrusion detection, especially when the ordering of traffic flows carries discriminative information [12]. Recent IoT and IIoT studies further extend this direction through deep transfer learning, multi-task multiview learning, and DDoS-specific deep models [20–22]. Squeeze-and-Excitation, residual connections, and attention mechanisms can further improve feature recalibration, gradient propagation, and temporal aggregation in deeper neural networks [23–25].

Graph neural networks provide another important direction because network traffic naturally forms a communication graph [26]. E-GraphSAGE incorporates edge features into graph-based intrusion detection [14, 15], while

dynamic graph modeling extends graph IDS to changing IoT environments [17]. Centrality-aware graph deep-learning frameworks have also been proposed for IoT intrusion detection, further supporting the usefulness of graph-structured traffic representations [27]. Nevertheless, graph-based models may suffer from oversmoothing or bottleneck effects when graph construction causes excessive neighborhood mixing [28, 29]. This motivates a flow-centered graph design and the use of attention-based embedding extraction.

Ensemble IDS approaches combine heterogeneous classifiers to improve robustness, following the broader success of bagging, boosting, and gradient-boosted decision trees [2, 30, 31]. xIDS integrates tree-based and neural models into an explainable ensemble framework [18], while recent stacking ensembles combine XGBoost, LightGBM, CatBoost, AdaBoost, and random forest meta-models [2, 19, 30, 31]. Recent studies in publisher venues also report subspace-clustering ensemble IDS designs and IoT-edge ensemble stacking, highlighting the continued relevance of ensemble fusion for practical intrusion detection [32, 33]. The proposed architecture follows this general ensemble motivation but focuses on explicitly fusing spatial graph information and temporal sequence information.

3. Proposed Method

3.1. Problem Formulation and Notation

Let a network-flow dataset be denoted by

$$\mathcal{D} = \{(f_i, \mathbf{x}_i, y_i, t_i, s_i, d_i, p_i)\}_{i=1}^N, \quad (1)$$

where f_i is the i -th flow, $\mathbf{x}_i \in \mathbb{R}^F$ is its normalized feature vector, $y_i \in \{1, \dots, C\}$ is the class label, t_i is the timestamp, s_i and d_i are the source and destination IP addresses, and p_i contains protocol and port information. The objective is to learn a classifier

$$\Phi : \mathbf{x}_i \mapsto \hat{\mathbf{p}}_i \in [0, 1]^C, \quad \hat{y}_i = \arg \max_{c \in \{1, \dots, C\}} \hat{p}_{i,c}, \quad (2)$$

that assigns each flow to either benign traffic or a specific intrusion class. The proposed method decomposes Φ into two complementary base learners and a meta-learner:

$$\Phi(f_i) = g_{\text{meta}} \left(\hat{\mathbf{p}}_i^{\text{gatxgb}} \parallel \hat{\mathbf{p}}_i^{\text{rcb}} \right), \quad (3)$$

where $\parallel$ denotes vector concatenation, $\hat{\mathbf{p}}_i^{\text{gatxgb}}$ is produced by the spatial GAT-XGBoost branch, and $\hat{\mathbf{p}}_i^{\text{rcb}}$ is produced by the temporal Residual CNN-BiLSTM branch.

3.2. Overall Architecture

The proposed framework contains two base learners and one meta-learner. The spatial branch constructs a flow-based graph and learns topology-aware embeddings using graph attention. These embeddings, optionally concatenated with raw normalized features, are classified by XGBoost. The temporal branch converts ordered flows into sliding windows, extracts local temporal patterns by residual convolutional blocks, and models long-range dependencies by BiLSTM attention. The meta-learner fuses the predictive distributions from both branches.

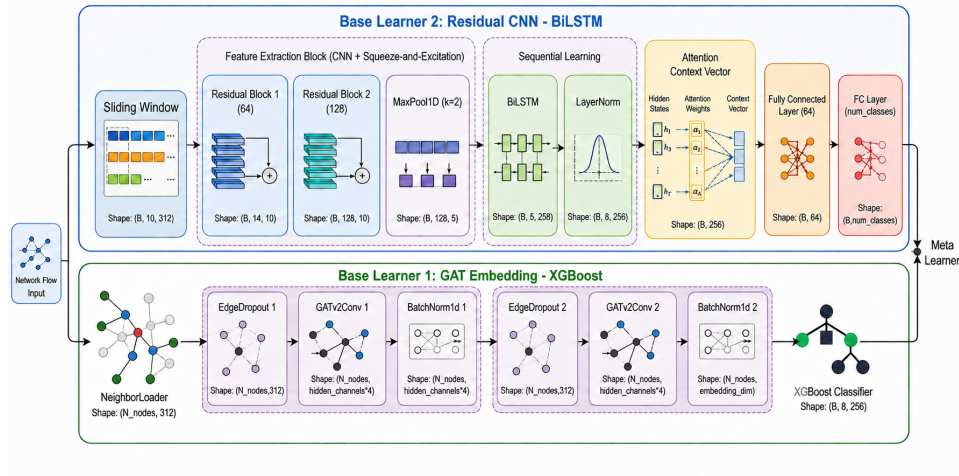


Figure 1: Overall architecture of the proposed ensemble intrusion detection framework.

3.3. Flow-Based Graph Construction

Instead of representing hosts as nodes and flows as edges, this study uses a flow-as-node graph. This design is motivated by dense IoT traffic, where a small number of hosts may exchange millions of flows. Compressing such flows into host-to-host edges can create information bottlenecks and can also make graph aggregation vulnerable to oversmoothing [28, 29]. In the proposed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, each node $v_i \in \mathcal{V}$ corresponds to one flow f_i , and its node feature is $\mathbf{x}_i$.

For each flow f_i , directed candidate edges are only created from historical flows to preserve causality and prevent temporal leakage. A directed edge (j, i) is added when

$$0 < t_i - t_j \leq \tau_{\max}, \quad \mathbb{I}(s_i = s_j \vee d_i = d_j) = 1, \quad (4)$$

and only the K nearest historical neighbors are kept:

$$\mathcal{N}_K(i) = \text{TopK}_j(-|t_i - t_j|), \quad K = 10. \quad (5)$$

Each edge is associated with a structural feature vector

$$\mathbf{e}_{ji} = [\Delta t_{ji}, \mathbb{I}(s_i = s_j \wedge d_i = d_j), \mathbb{I}(\text{sport}_i = \text{sport}_j), \mathbb{I}(\text{dport}_i = \text{dport}_j)], \quad (6)$$

where $\Delta t_{ji} = t_i - t_j$. Algorithm 1 summarizes the construction procedure.

Algorithm 1: Pseudo-code for flow-based graph construction.

Input: Chronologically unordered flows $\mathcal{F}$, time threshold $\tau_{\max}$, maximum degree K .

Output: Directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with edge features.

1. Sort all flows by timestamp so that $t_1 \leq t_2 \leq \dots \leq t_N$.
 2. Create one node v_i for each flow f_i and assign node feature $\mathbf{x}_i$.
 3. For each current flow f_i , scan only historical candidates f_j with $j < i$.
 4. Keep candidate f_j if $0 < t_i - t_j \leq \tau_{\max}$ and the two flows share source or destination IP.
 5. Rank candidates by increasing $|t_i - t_j|$ and retain at most K neighbors.
 6. Add directed edge (j, i) for each retained neighbor and compute $\mathbf{e}_{ji}$.
 7. Return $\mathcal{G}$ for GAT embedding learning.
-

3.4. Base Learner 1: GAT Embedding - XGBoost

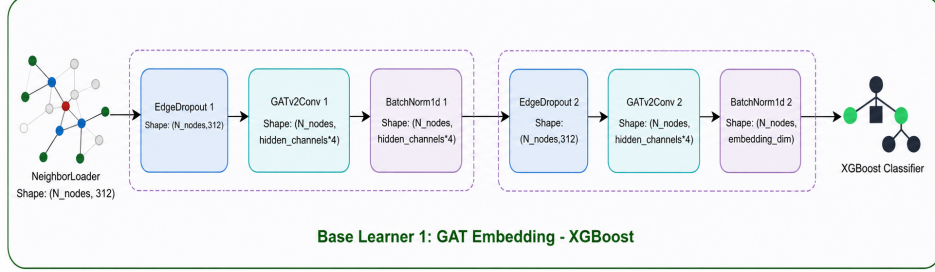


Figure 2: Spatial base learner based on GAT embeddings and XGBoost classification.

Given the constructed graph, the GAT branch computes an embedding $\mathbf{h}_i$ for each flow node. The initial hidden state is

$$\mathbf{h}_i^{(0)} = \mathbf{W}_0 \mathbf{x}_i + \mathbf{b}_0. \quad (7)$$

Following the original Graph Attention Network formulation [1], for a GATv2-style attention layer [34], the unnormalized attention score from neighbor j to node i is

$$e_{ij}^{(m)} = \mathbf{a}_m^\top \text{LeakyReLU} \left(\mathbf{W}_m [\mathbf{h}_i^{(\ell)} \parallel \mathbf{h}_j^{(\ell)} \parallel \mathbf{e}_{ji}] \right), \quad (8)$$

where m indexes the attention head. The normalized attention coefficient is

$$\alpha_{ij}^{(m)} = \frac{\exp(e_{ij}^{(m)})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik}^{(m)})}. \quad (9)$$

The node representation is updated by aggregating the neighbor messages:

$$\mathbf{h}_i^{(\ell+1)} = \bigoplus_{m=1}^M \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(m)} \mathbf{W}_m \mathbf{h}_j^{(\ell)} \right), \quad (10)$$

where M is the number of attention heads and $\sigma(\cdot)$ is a nonlinear activation function. After the final graph layer, the learned embedding is denoted by $\mathbf{z}_i = \mathbf{h}_i^{(L)}$.

Two spatial variants are evaluated. The embedding-only variant uses $\mathbf{z}_i$ as input to XGBoost:

$$\hat{\mathbf{p}}_i^{\text{gatxgb}} = \text{XGBoost}(\mathbf{z}_i). \quad (11)$$

The best-performing variant concatenates graph embeddings with raw normalized features:

$$\tilde{\mathbf{x}}_i = \mathbf{z}_i \parallel \mathbf{x}_i, \quad \hat{\mathbf{p}}_i^{\text{gatrawxgb}} = \text{XGBoost}(\tilde{\mathbf{x}}_i). \quad (12)$$

This design lets the branch exploit both topology-aware representations and original tabular traffic attributes. For the multi-class XGBoost classifier, the additive tree model is written as

$$\hat{\mathbf{p}}_i = \text{Softmax} \left(\sum_{q=1}^Q \mathbf{f}_q(\tilde{\mathbf{x}}_i) \right), \quad \mathbf{f}_q \in \mathcal{F}_{\text{tree}}, \quad (13)$$

and the regularized training objective is

$$\mathcal{L}_{\text{xgb}} = \sum_{i=1}^N \ell(y_i, \hat{\mathbf{p}}_i) + \sum_{q=1}^Q \Omega(\mathbf{f}_q), \quad \Omega(\mathbf{f}) = \gamma T_{\mathbf{f}} + \frac{1}{2} \lambda \|\mathbf{w}_{\mathbf{f}}\|_2^2, \quad (14)$$

where $T_{\mathbf{f}}$ is the number of leaves and $\mathbf{w}_{\mathbf{f}}$ contains leaf weights. This objective is useful for tabular IDS data because it balances fitting capacity with explicit complexity control.

3.5. Base Learner 2: Residual CNN-BiLSTM

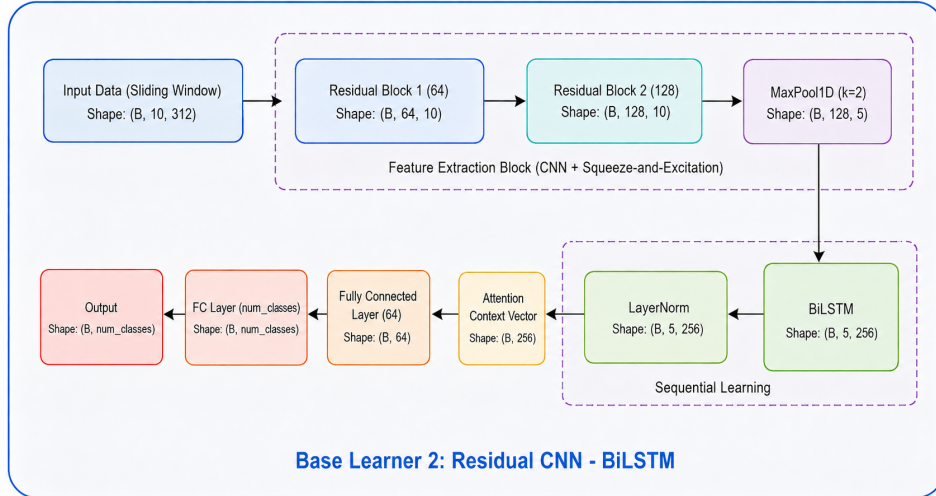


Figure 3: Temporal base learner based on residual CNN blocks and BiLSTM attention.

The temporal branch starts from chronologically ordered flows and constructs fixed-length windows of size T :

$$\mathbf{S}_i = [\mathbf{x}_{i-T+1}, \mathbf{x}_{i-T+2}, \dots, \mathbf{x}_i] \in \mathbb{R}^{T \times F}. \quad (15)$$

Each window is assigned the label of its last flow, which makes the prediction causal with respect to the current flow. A one-dimensional convolution extracts local temporal features:

$$\mathbf{U} = \text{Conv1D}(\mathbf{S}_i; \mathbf{W}_{\text{conv}}) + \mathbf{b}_{\text{conv}}. \quad (16)$$

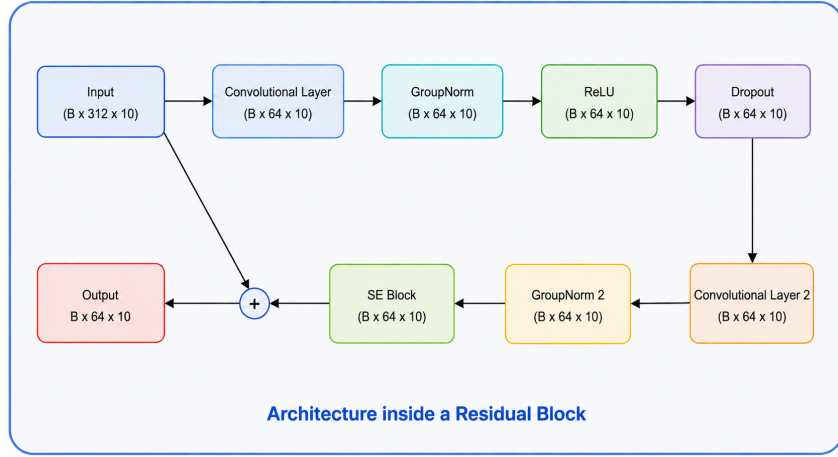


Figure 4: Residual block used in the temporal base learner.

Figure 4 illustrates the internal structure of the residual block before the block is formalized mathematically. A residual block is defined as

$$\mathbf{R} = \text{ReLU}(\mathcal{F}_{\text{res}}(\mathbf{U}) + \mathcal{P}(\mathbf{U})), \quad (17)$$

where $\mathcal{F}_{\text{res}}(\cdot)$ is a stack of convolution, normalization, and activation operations, and $\mathcal{P}(\cdot)$ is either the identity mapping or a projection used to match channel dimensions. Each residual block also contains a Squeeze-and-Excitation recalibration unit. Given an intermediate feature map $\mathbf{U} \in \mathbb{R}^{C \times T}$, channel-wise squeeze statistics are computed by

$$z_c = \frac{1}{T} \sum_{t=1}^T U_{c,t}, \quad c = 1, \dots, C. \quad (18)$$

The excitation vector is then generated by

$$\mathbf{s} = \sigma(\mathbf{W}_2 \delta(\mathbf{W}_1 \mathbf{z})), \quad (19)$$

where $\delta(\cdot)$ is ReLU and $\sigma(\cdot)$ is the sigmoid function. The recalibrated output is

$$\tilde{U}_{c,t} = s_c U_{c,t}. \quad (20)$$

This mechanism adaptively emphasizes informative temporal channels and suppresses noisy channels. The two residual blocks used in this branch are configured as summarized in Table 2.

Table 2: Configuration of residual blocks in the temporal base learner.

Parameter	Residual Block 1	Residual Block 2
Input channels	F	64
Output channels	64	128
Kernel size	3	3
Padding	1	1
Group normalization	8 groups	8 groups
SE reduction ratio	8	8
Dropout rate	0.2	0.2
Output shape	$B \times 64 \times T$	$B \times 128 \times T$

After temporal pooling, the CNN representation is transposed into a sequence $\{\bar{\mathbf{x}}_t\}_{t=1}^{T'}$ and processed by a bidirectional LSTM:

$$\vec{\mathbf{h}}_t = \text{LSTM}_{\text{fw}}(\bar{\mathbf{x}}_t, \vec{\mathbf{h}}_{t-1}), \quad (21)$$

$$\overleftarrow{\mathbf{h}}_t = \text{LSTM}_{\text{bw}}(\bar{\mathbf{x}}_t, \overleftarrow{\mathbf{h}}_{t+1}), \quad (22)$$

$$\mathbf{h}_t = \vec{\mathbf{h}}_t \parallel \overleftarrow{\mathbf{h}}_t. \quad (23)$$

Attention scores and weights are computed as

$$e_t = \mathbf{v}_{\text{att}}^\top \tanh(\mathbf{W}_{\text{att}} \mathbf{h}_t + \mathbf{b}_{\text{att}}), \quad \alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^{T'} \exp(e_j)}. \quad (24)$$

The final temporal context vector is

$$\mathbf{c} = \sum_{t=1}^{T'} \alpha_t \mathbf{h}_t. \quad (25)$$

The detailed BiLSTM-attention configuration used after the residual blocks is summarized in Table 3.

Parameter	Value
Input tensor shape	$B \times T' \times 128$ ($T' = 5$)
Hidden size	128
BiLSTM output dimension	256 (128×2)
Attention activation	Tanh
Normalization layer	LayerNorm with 256 units
Context vector shape	$B \times 256$

The branch prediction is generated by a fully connected classifier with dropout and layer normalization:

$$\hat{\mathbf{p}}_i^{\text{rcb}} = \text{Softmax}(\mathbf{W}_2 \text{Dropout}(\text{ReLU}(\text{LayerNorm}(\mathbf{W}_1 \mathbf{c} + \mathbf{b}_1))) + \mathbf{b}_2). \quad (26)$$

3.6. Meta-Learner

The meta-learner receives the class-probability vectors from the two base learners:

$$\mathbf{m}_i = \hat{\mathbf{p}}_i^{\text{gatraxgb}} \parallel \hat{\mathbf{p}}_i^{\text{rcb}} \in \mathbb{R}^{2C}. \quad (27)$$

Two meta-classifier variants are evaluated. The logistic-regression meta-learner computes

$$\hat{\mathbf{p}}_i^{\text{final}} = \text{Softmax}(\mathbf{W}_{\text{LR}} \mathbf{m}_i + \mathbf{b}_{\text{LR}}), \quad (28)$$

whereas the XGBoost meta-learner uses boosted decision trees [2]:

$$\hat{\mathbf{p}}_i^{\text{final}} = \text{XGBoost}_{\text{meta}}(\mathbf{m}_i). \quad (29)$$

The final predicted label is $\hat{y}_i = \arg \max_c \hat{p}_{i,c}^{\text{final}}$.

4. Experimental Setup

4.1. Datasets

The evaluation uses three use cases from two recent intrusion detection datasets. Recent IDS benchmarks such as UNSW-NB15, CICIDS2017, Bot-IoT, and ToN-IoT have shown the importance of realistic traffic diversity,

attack variety, and reproducible evaluation protocols [35–38]. The first two use cases are derived from IoT-IDS-Zwave-2025, a Z-Wave IoT intrusion detection dataset designed for smart-device profiling and attack analysis [3]. Use Case 1 focuses on HTTP-based DDoS attacks and contains 2,269,748 flow samples across multiple DDoS attack classes. Use Case 2 focuses on Application-based DDoS variants and contains 10,946,824 flow samples. The third use case uses CIC IIoT 2025, a benchmark for Industrial IoT attack analysis [4]. The experiments use the 1-second aggregation window, yielding 227,191 samples grouped into seven attack categories or benign traffic.

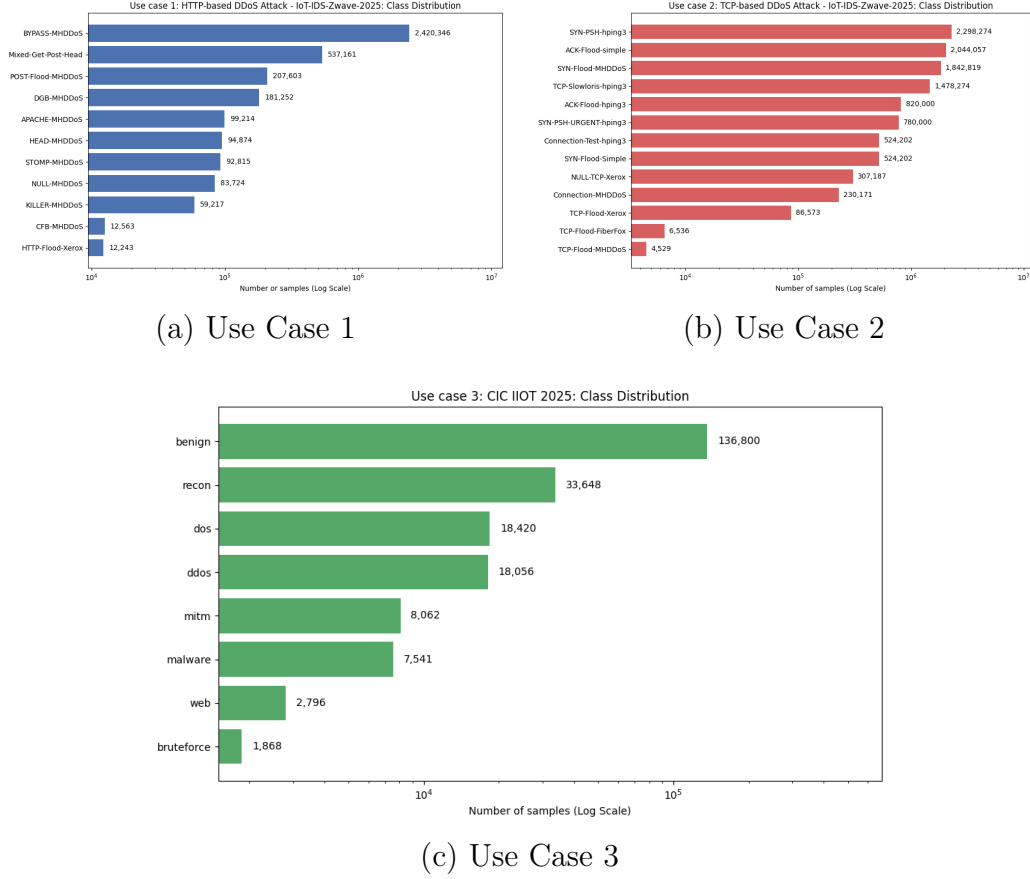


Figure 5: Class distributions for the three evaluation use cases.

4.2. Preprocessing and Evaluation Protocol

All datasets are processed through the same preprocessing pipeline. Missing values are imputed, categorical attributes are one-hot encoded, target

labels are encoded, and numerical features are normalized using a QuantileTransformer. To reduce temporal data leakage, each dataset is ordered chronologically before splitting. A hybrid stratified and time-based split is then applied so that class balance is preserved while temporal continuity remains explicit. The train/validation/test ratios are 65:15:20 for Use Case 1, 30:40:30 for Use Case 2, and 70:15:15 for Use Case 3.

The models are evaluated using macro precision, macro recall, macro F1-score, and AUC-ROC. Macro-averaged metrics are emphasized because the datasets are strongly imbalanced and minority attack classes are critical in intrusion detection, a setting closely related to imbalanced-learning challenges. Each standalone base model is trained and evaluated three times, and the reported results are averaged to reduce variance.

5. Results and Discussion

In this section, the complete quantitative results are illustrated. The discussion focuses on the major empirical patterns: the graph-XGBoost branch is strongest on structured flow relations, the Residual CNN-BiLSTM branch is more robust on temporally complex traffic, and the meta-learner benefits from combining both representations.

5.1. Ablation Study

Tables 4–6 retain all ablation results. Generally, the Residual CNN-BiLSTM ablation shows that attention, sliding windows, and residual blocks all contribute to performance, especially under imbalance and temporal drift. The GAT/XGBoost ablation shows that graph embeddings are useful, while concatenating raw features can improve stability. The meta-learner ablation shows that both logistic regression and XGBoost are viable fusion strategies, with the best choice depending on the use case.

Table 4: Ablation Study Results for Residual CNN-BiLSTM Variations across Three Use Cases

Variations	Macro Precision	Macro Recall	Macro F1
Use Case 1			
Proposed (US+RB+SW+Base+Att)	0.8695 $\pm$ 0.0160	0.9184 $\pm$ 0.0033	0.8794 $\pm$ 0.0138
w/o Att (US+RB+SW+Base)	0.8352 $\pm$ 0.0117	0.8936 $\pm$ 0.0083	0.8416 $\pm$ 0.0152
w/o SW (US+RB+Base+Att)	0.7648 $\pm$ 0.0371	0.7991 $\pm$ 0.0422	0.7624 $\pm$ 0.0455
w/o RB (US+SW+Base+Att)	0.8371 $\pm$ 0.0101	0.8911 $\pm$ 0.0119	0.8425 $\pm$ 0.0144
w/o RB, SW (US+Base+Att) [12]	0.7584 $\pm$ 0.0328	0.7865 $\pm$ 0.0266	0.7505 $\pm$ 0.0394
w/o RB, SW (HS+Base+Att)	0.7921 $\pm$ 0.0134	0.8582 $\pm$ 0.0003	0.8044 $\pm$ 0.0072
Use Case 2			
Proposed (US+RB+SW+Base+Att)	0.9218 $\pm$ 0.0151	0.9581 $\pm$ 0.0021	0.9333 $\pm$ 0.0105
w/o Att (US+RB+SW+Base)	0.8805 $\pm$ 0.0517	0.9153 $\pm$ 0.0350	0.8886 $\pm$ 0.0429
w/o SW (US+RB+Base+Att)	0.9209 $\pm$ 0.0136	0.9392 $\pm$ 0.0117	0.9197 $\pm$ 0.0151
w/o RB (US+SW+Base+Att)	0.8277 $\pm$ 0.0047	0.8869 $\pm$ 0.0029	0.8411 $\pm$ 0.0040
w/o RB, SW (US+Base+Att) [12]	0.9040 $\pm$ 0.0096	0.9401 $\pm$ 0.0042	0.9110 $\pm$ 0.0062
w/o RB, SW (HS+Base+Att)	-	-	-
Use Case 3			
Proposed (US+RB+SW+Base+Att)	0.7593 $\pm$ 0.0081	0.7459 $\pm$ 0.0321	0.7182 $\pm$ 0.0116
w/o Att (US+RB+SW+Base)	0.7011 $\pm$ 0.0097	0.6670 $\pm$ 0.0060	0.6542 $\pm$ 0.0025
w/o SW (US+RB+Base+Att)	0.6234 $\pm$ 0.0188	0.5838 $\pm$ 0.0057	0.5907 $\pm$ 0.0117
w/o RB (US+SW+Base+Att)	0.7469 $\pm$ 0.0202	0.7113 $\pm$ 0.0486	0.6971 $\pm$ 0.0335
w/o RB, SW (US+Base+Att) [12]	0.5998 $\pm$ 0.0127	0.5852 $\pm$ 0.0053	0.5792 $\pm$ 0.0072
w/o RB, SW (HS+Base+Att)	0.6480 $\pm$ 0.0248	0.6250 $\pm$ 0.0109	0.6172 $\pm$ 0.0077

Note: **Base:** CNN + LSTM; **US:** UnderSampling; **HS:** Hybrid Sampling (SMOTE + UnderSampling); **RB:** Residual Block; **SW:** Sliding Window; **Att:** Attention; **w/o:** Without.

Table 4 shows that the full Residual CNN–BiLSTM configuration consistently achieves the best Macro F1 across all three use cases. Removing sliding windows causes a particularly large degradation in Use Case 1 and Use Case 3, indicating that temporal context is essential for distinguishing evolving attack patterns. The attention mechanism and residual blocks also provide stable gains, suggesting that both time-step selection and deeper feature extraction contribute to the robustness of the temporal branch. Compared with the attention-based CNN–BiLSTM architecture proposed for efficient IoT intrusion detection [12], the proposed base learner achieves higher Macro F1 in all three use cases, confirming the added value of residual feature extraction and sliding-window temporal modeling.

Table 5: Ablation Study Results for GAT and XGBoost Architectures across Three Use Cases

Model	Macro Precision	Macro Recall	Macro F1
Use Case 1			
GAT Embedding + Raw Features + XGBoost (Proposed)	0.9333 $\pm$ 0.0018	0.9162 $\pm$ 0.0040	0.9205 $\pm$ 0.0020
GAT Embedding + XGBoost (Proposed)	0.9287 $\pm$ 0.0085	0.9210 $\pm$ 0.0083	0.9202 $\pm$ 0.0085
GAT Only [1]	0.8797 $\pm$ 0.0164	0.9210 $\pm$ 0.0053	0.8870 $\pm$ 0.0197
XGBoost Only	0.8445 $\pm$ 0.0000	0.8788 $\pm$ 0.0000	0.8501 $\pm$ 0.0000
Use Case 2			
GAT Embedding + Raw Features + XGBoost (Proposed)	0.9504 $\pm$ 0.0058	0.9682 $\pm$ 0.0033	0.9544 $\pm$ 0.0057
GAT Embedding + XGBoost (Proposed)	0.9385 $\pm$ 0.0146	0.9502 $\pm$ 0.0224	0.9369 $\pm$ 0.0229
GAT Only [1]	0.9497 $\pm$ 0.0009	0.9682 $\pm$ 0.0017	0.9548 $\pm$ 0.0007
XGBoost Only	0.9531 $\pm$ 0.0000	0.9668 $\pm$ 0.0000	0.9568 $\pm$ 0.0000
Use Case 3			
GAT Embedding + Raw Features + XGBoost (Proposed)	0.7439 $\pm$ 0.0080	0.6531 $\pm$ 0.0042	0.6809 $\pm$ 0.0047
GAT Embedding + XGBoost (Proposed)	0.7439 $\pm$ 0.0076	0.6540 $\pm$ 0.0060	0.6835 $\pm$ 0.0065
GAT Only [1]	0.7463 $\pm$ 0.0029	0.6382 $\pm$ 0.0057	0.6661 $\pm$ 0.0060
XGBoost Only	0.6678 $\pm$ 0.0059	0.5993 $\pm$ 0.0018	0.6116 $\pm$ 0.0043

Table 5 demonstrates that GAT embeddings improve the discriminative capability of XGBoost compared with using XGBoost alone in Use Case 1 and Use Case 3. The raw-feature concatenation variant is especially effective in Use Case 1, where combining topology-aware embeddings with original flow statistics yields the highest Macro F1. In Use Case 2, XGBoost-only remains highly competitive, implying that this use case contains class boundaries that are already well captured by tabular flow attributes.

Table 6: Ablation Study Results for Meta-Learner Architectures across Two Use Cases

Model	Macro Precision	Macro Recall	Macro F1
Use Case 1			
Proposed Meta-Learner (LR)	0.9630	0.9654	0.9638
Proposed Meta-Learner (XGBoost)	0.9705	0.9702	0.9697
Base: Residual CNN-BiLSTM	0.8985	0.9324	0.9028
Base: GAT + XGBoost	0.9706	0.9659	0.9672
Use Case 2			
Proposed Meta-Learner (LR)	0.9554	0.9665	0.9571
Proposed Meta-Learner (XGBoost)	0.9456	0.9700	0.9527
Base: Residual CNN-BiLSTM	0.9454	0.9647	0.9471
Base: GAT + XGBoost	0.9502	0.9620	0.9523

Note: **Proposed Meta-Learner** combines spatial features (GAT Embedding + Raw + XGBoost) and temporal features (Residual CNN-BiLSTM). **LR**: Logistic Regression meta-classifier; **XGBoost**: XGBoost meta-classifier; **Base**: Independent base learners before ensemble.

Table 6 confirms the benefit of fusing spatial and temporal predictions through a meta-learner. In Use Case 1, the XGBoost meta-learner achieves the strongest Macro F1, showing that nonlinear fusion can exploit interactions between the two base learners. In Use Case 2, logistic regression performs slightly better in Macro F1, suggesting that a simpler linear fusion rule is sufficient when the base predictions are already well separated.

5.2. Base Learner Results

Tables 7–9 report the complete base-learner comparison across the three use cases. In Use Case 1, the proposed GAT Embedding + Raw + XGBoost model achieves the best macro F1 and AUC-ROC, while the embedding-only variant is nearly identical. In Use Case 2, tree-based and graph-enhanced learners remain very competitive because the class boundaries are well represented by flow statistics. In Use Case 3, the Residual CNN-BiLSTM branch becomes the strongest proposed base learner, confirming the importance of temporal modeling under more complex IIoT traffic. Notably, given the novelty of the employed benchmark datasets, Dynamic IDS [17] and E-GraphSAGE [15] were reconstructed with minor input-dimension adapta-

tions, while preserving their original graph-based modeling logic.

Table 7: Performance Evaluation of Intrusion Detection Models - Use Case 1

Model	Macro Precision	Macro Recall	Macro F1	AUC-ROC
GAT Embedding + Raw + XGBoost (Proposed)	0.9333 $\pm$ 0.0018	0.9162 $\pm$ 0.0040	0.9205 $\pm$ 0.0020	0.9984 $\pm$ 0.0003
GAT Embedding + XGBoost (Proposed)	0.9287 $\pm$ 0.0085	0.9210 $\pm$ 0.0083	0.9202 $\pm$ 0.0085	0.9983 $\pm$ 0.0003
Residual CNN BiLSTM (Proposed)	0.8695 $\pm$ 0.0160	0.9184 $\pm$ 0.0033	0.8794 $\pm$ 0.0138	0.9906 $\pm$ 0.0053
SVM	0.7635 $\pm$ 0.0325	0.7070 $\pm$ 0.0312	0.6858 $\pm$ 0.0172	0.9802 $\pm$ 0.0022
KNN	0.8057 $\pm$ 0.0000	0.7578 $\pm$ 0.0000	0.7472 $\pm$ 0.0000	-
Logistic Regression	0.7429 $\pm$ 0.0286	0.7253 $\pm$ 0.0137	0.6979 $\pm$ 0.0191	0.9855 $\pm$ 0.0037
Random Forest	0.8433 $\pm$ 0.0027	0.8848 $\pm$ 0.0005	0.8539 $\pm$ 0.0017	0.9337 $\pm$ 0.0000
Decision Tree	0.8483 $\pm$ 0.0029	0.8874 $\pm$ 0.0042	0.8580 $\pm$ 0.0029	0.9269 $\pm$ 0.0000
XGBoost [2]	0.8445 $\pm$ 0.0000	0.8788 $\pm$ 0.0000	0.8501 $\pm$ 0.0000	0.9967 $\pm$ 0.0000
GAT [1]	0.8797 $\pm$ 0.0164	0.9210 $\pm$ 0.0053	0.8870 $\pm$ 0.0197	0.9715 $\pm$ 0.0037
Flow-based Dynamic IDS [17]	0.8594 $\pm$ 0.0119	0.9130 $\pm$ 0.0071	0.8700 $\pm$ 0.0076	0.9970 $\pm$ 0.0010
E-GraphSAGE [15]	0.7502 $\pm$ 0.0572	0.8024 $\pm$ 0.0255	0.7391 $\pm$ 0.0590	0.9499 $\pm$ 0.0256

Table 7 shows that the proposed GAT Embedding + Raw + XGBoost model achieves the best Macro F1 and AUC-ROC on the HTTP-based attack use case. The embedding-only GAT + XGBoost variant performs nearly identically, confirming that graph attention embeddings provide a highly discriminative spatial representation. Compared with Dynamic IDS and E-GraphSAGE, the proposed spatial branch is more stable, while the Residual CNN-BiLSTM branch still maintains strong recall due to its temporal modeling capability.

Table 8: Performance Evaluation of Intrusion Detection Models - Use Case 2

Model	Macro Precision	Macro Recall	Macro F1	AUC-ROC
GAT Embedding + Raw + XGBoost (Proposed)	0.9504 $\pm$ 0.0058	0.9682 $\pm$ 0.0033	0.9544 $\pm$ 0.0057	0.9987 $\pm$ 0.0007
GAT Embedding + XGBoost (Proposed)	0.9385 $\pm$ 0.0146	0.9502 $\pm$ 0.0224	0.9369 $\pm$ 0.0229	0.9968 $\pm$ 0.0005
Residual CNN BiLSTM (Proposed)	0.9218 $\pm$ 0.0151	0.9581 $\pm$ 0.0021	0.9333 $\pm$ 0.0105	0.9996 $\pm$ 0.0001
SVM	0.9264 $\pm$ 0.0115	0.8780 $\pm$ 0.0050	0.8772 $\pm$ 0.0033	0.9987 $\pm$ 0.0003
KNN	-	-	-	-
Logistic Regression	0.9188 $\pm$ 0.0288	0.9022 $\pm$ 0.0315	0.8878 $\pm$ 0.0348	0.9978 $\pm$ 0.0009
Random Forest	0.8159 $\pm$ 0.0000	0.9321 $\pm$ 0.0000	0.8251 $\pm$ 0.0000	0.9392 $\pm$ 0.0000
Decision Tree	0.8126 $\pm$ 0.0000	0.8687 $\pm$ 0.0000	0.8160 $\pm$ 0.0000	0.8805 $\pm$ 0.0000
XGBoost [2]	0.9531 $\pm$ 0.0000	0.9668 $\pm$ 0.0000	0.9568 $\pm$ 0.0000	0.9998 $\pm$ 0.0000
GAT [1]	0.9497 $\pm$ 0.0009	0.9682 $\pm$ 0.0017	0.9548 $\pm$ 0.0007	0.9996 $\pm$ 0.0000
Flow-based Dynamic IDS [17]	0.9188 $\pm$ 0.0163	0.9582 $\pm$ 0.0050	0.9256 $\pm$ 0.0155	0.9995 $\pm$ 0.0004
E-GraphSAGE [15]	0.7441 $\pm$ 0.0482	0.8330 $\pm$ 0.0287	0.7651 $\pm$ 0.0409	0.9778 $\pm$ 0.0132

Table 8 indicates that Use Case 2 is more favorable to tree-based and graph-enhanced learners. XGBoost alone obtains the highest Macro F1, while GAT and the proposed GAT-based variants remain very close, sug-

gesting that the application-based DDoS classes are strongly encoded in flow-level tabular features. Nevertheless, the Residual CNN-BiLSTM branch also achieves competitive AUC-ROC, showing that temporal features still provide useful complementary information.

Table 9: Performance Evaluation of Intrusion Detection Models - Use Case 3

Model	Macro Precision	Macro Recall	Macro F1	AUC-ROC
GAT Embedding + Raw + XGBoost (Proposed)	0.7439 $\pm$ 0.0080	0.6531 $\pm$ 0.0042	0.6809 $\pm$ 0.0047	0.9054 $\pm$ 0.0037
GAT Embedding + XGBoost (Proposed)	0.7439 $\pm$ 0.0076	0.6540 $\pm$ 0.0060	0.6835 $\pm$ 0.0065	0.9016 $\pm$ 0.0062
Residual CNN BiLSTM (Proposed)	0.7593 $\pm$ 0.0081	0.7459 $\pm$ 0.0321	0.7182 $\pm$ 0.0116	0.9558 $\pm$ 0.0125
SVM	0.6340 $\pm$ 0.0201	0.5741 $\pm$ 0.0120	0.5786 $\pm$ 0.0211	0.9164 $\pm$ 0.0030
KNN	0.4974 $\pm$ 0.0000	0.4771 $\pm$ 0.0000	0.4317 $\pm$ 0.0000	0.7325 $\pm$ 0.0000
Logistic Regression	0.6327 $\pm$ 0.0583	0.5743 $\pm$ 0.0294	0.5783 $\pm$ 0.0391	0.9204 $\pm$ 0.0017
Random Forest	0.6477 $\pm$ 0.0004	0.6190 $\pm$ 0.0001	0.6217 $\pm$ 0.0001	0.7975 $\pm$ 0.0000
Decision Tree	0.6426 $\pm$ 0.0011	0.5787 $\pm$ 0.0010	0.5987 $\pm$ 0.0012	0.7739 $\pm$ 0.0000
XGBoost [2]	0.6678 $\pm$ 0.0059	0.5993 $\pm$ 0.0018	0.6116 $\pm$ 0.0043	0.9241 $\pm$ 0.0000
GAT [1]	0.7463 $\pm$ 0.0029	0.6382 $\pm$ 0.0057	0.6661 $\pm$ 0.0060	0.9546 $\pm$ 0.0052
Flow-based Dynamic IDS [17]	-	-	-	-
E-GraphSAGE [15]	-	-	-	-

Table 9 highlights the difficulty of the CIC IIoT 2025 use case, where most models experience a clear performance drop. The Residual CNN-BiLSTM branch achieves the best Macro F1 and AUC-ROC, indicating that temporal dependencies are more important for this industrial IoT traffic scenario. In contrast, the graph-based branches remain competitive in precision but are less effective in recall, suggesting that spatial relations alone are insufficient for this more complex setting.

5.3. Comparison with Ensemble IDS Baselines

Tables 10–12 compare the proposed meta-learners with recent ensemble IDS baselines. The comparison focuses on whether stacking the spatial GAT–XGBoost branch and the temporal Residual CNN–BiLSTM branch can provide more robust predictions than ensembles composed mainly of tree-based classifiers. Overall, the proposed ensemble is strongest when the two base learners provide complementary evidence, especially in cases where traffic behavior cannot be fully captured by tabular flow features alone.

Table 10: Performance comparison of Ensemble Learning models - Use Case 1

Model	Macro Precision	Macro Recall	Macro F1	AUC-ROC
Proposed Meta-Learner (LR)	0.9630	0.9654	0.9638	0.9995
Proposed Meta-Learner (XGBoost)	0.9705	0.9702	0.9697	0.9997
xIDS [18]	0.8883	0.9127	0.8967	0.9984
XGBoost, LightGBM, CatBoost, AdaBoost [19]	0.8985	0.9130	0.9028	0.9986

Note: **Proposed Meta-Learner** combines spatial features (GAT Embedding + Raw + XGBoost) and temporal features (Residual CNN-BiLSTM). **LR**: Logistic Regression meta-classifier; **XGBoost**: XGBoost meta-classifier.

Table 10 shows that the proposed XGBoost meta-learner achieves the best results across all evaluation metrics in Use Case 1. Compared with xIDS and the XGBoost–LightGBM–CatBoost–AdaBoost ensemble, the proposed model improves Macro F1 by a clear margin while also maintaining an almost perfect AUC-ROC. This result indicates that HTTP-based DDoS traffic benefits from combining spatial graph-derived predictions with temporal sequence predictions. The logistic-regression meta-learner is also strong, but the XGBoost meta-learner provides a small additional gain, suggesting that nonlinear fusion is useful when both base learners produce highly informative but not identical decision patterns.

Table 11: Performance comparison of Ensemble Learning models - Use Case 2

Model	Macro Precision	Macro Recall	Macro F1	AUC-ROC
Proposed Meta-Learner (LR)	0.9554	0.9665	0.9571	0.9981
Proposed Meta-Learner (XGBoost)	0.9456	0.9700	0.9527	0.9848
xIDS [18]	0.9735	0.9747	0.9735	0.9982
XGBoost, LightGBM, CatBoost, AdaBoost [19]	0.9698	0.9573	0.9619	0.9998

Note: **Proposed Meta-Learner** combines spatial features (GAT Embedding + Raw + XGBoost) and temporal features (Residual CNN-BiLSTM). **LR**: Logistic Regression meta-classifier; **XGBoost**: XGBoost meta-classifier.

Table 11 presents a different trend. The xIDS baseline obtains the highest Macro Precision, Macro Recall, and Macro F1, while the XGBoost–LightGBM–CatBoost–AdaBoost ensemble achieves the highest AUC-ROC. The proposed meta-learners remain competitive, but they do not surpass the strongest tree-based ensembles in this use case. This suggests that application-based DDoS attacks in Use Case 2 are already well separated by conventional flow-level feature interactions, making the additional spatial–

temporal fusion less beneficial. Nevertheless, the logistic-regression meta-learner achieves a higher Macro F1 than the proposed XGBoost meta-learner, indicating that simpler and more stable fusion can be preferable when the base-learner predictions are already well calibrated.

Table 12: Performance comparison of Ensemble Learning models - Use Case 3

Model	Macro Precision	Macro Recall	Macro F1	AUC-ROC
Proposed Meta-Learner (LR)	0.8040	0.7951	0.7505	0.9449
Proposed Meta-Learner (XGBoost)	0.8310	0.7405	0.7490	0.9844
xIDS [18]	0.7542	0.6640	0.6760	0.9022
XGBoost, LightGBM, CatBoost, AdaBoost [19]	0.8154	0.6690	0.6992	0.9464

Note: **Proposed Meta-Learner** combines spatial features (GAT Embedding + Raw + XGBoost) and temporal features (Residual CNN-BiLSTM). **LR**: Logistic Regression meta-classifier; **XGBoost**: XGBoost meta-classifier.

Table 12 demonstrates the main robustness advantage of the proposed ensemble in the most challenging setting. Both proposed meta-learners outperform the baseline ensembles in Macro F1, and the XGBoost meta-learner achieves the highest Macro Precision and AUC-ROC. The logistic-regression meta-learner obtains the highest Macro Recall and Macro F1, which indicates better recovery of minority or harder-to-detect attack classes. These results suggest that, for industrial IoT traffic, the complementary information from spatial graph representations and temporal sequence representations helps reduce the limitations of purely tree-based ensemble baselines.

Overall, the ensemble comparison confirms that the proposed architecture is not uniformly dominant on every metric, but it provides consistently strong and robust performance across heterogeneous IoT and IIoT settings. The XGBoost meta-learner is more effective when nonlinear interactions between the two branches are important, whereas the logistic-regression meta-learner can be more stable when recall and class-balance robustness are critical. These findings support the central motivation of the proposed framework: spatial graph representations and temporal sequence representations provide complementary signals for intrusion attack classification.

6. Case Study: Real-Time Deployment

To complement the offline evaluation, a deployment case study is implemented to examine whether the proposed ensemble NIDS can operate

within a near real-time security-monitoring workflow. The objective of this case study is not to introduce an additional benchmark, but to demonstrate that the trained spatial-temporal ensemble can be embedded into a practical streaming pipeline. The deployment therefore focuses on the end-to-end data lifecycle, including traffic generation, message buffering, stream preprocessing, model inference, time-series storage, and dashboard visualization.

Figure 6 illustrates the prototype deployment architecture. A data generator replays preprocessed network-flow records in chronological order to simulate continuous traffic arrival. Apache Kafka is used as the message broker to decouple data ingestion from downstream processing, while Spark Structured Streaming consumes the raw events and applies the same feature-transformation logic used during model training. The inference daemon then receives the processed flows, constructs the required spatial and temporal inputs, and executes the trained ensemble model to produce predicted attack labels and confidence scores. These outputs, together with timestamps and flow metadata, are stored in InfluxDB for time-series querying.

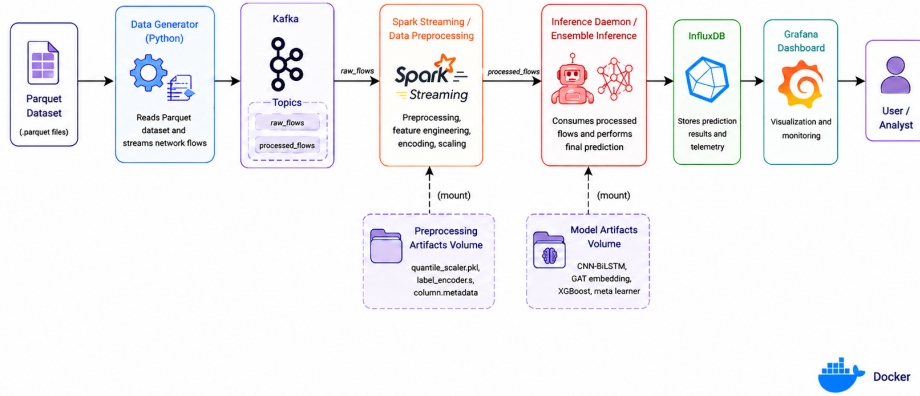


Figure 6: Deployment diagram of the near real-time ensemble NIDS prototype.

Grafana is used as the visualization layer of the deployment. As shown in Figure 7, the dashboard summarizes the operational status and security outputs of the streaming NIDS. It reports the total number of inferred flows, the proportion of benign and malicious traffic, the distribution of detected

attack categories, and the temporal evolution of threats. These panels allow administrators to quickly verify whether the pipeline is actively processing traffic, identify dominant threat families, and observe bursty malicious behavior over time.



Figure 7: Grafana dashboard for monitoring inferred flows and detected attack patterns.

Overall, the case study demonstrates the operational feasibility of the proposed architecture beyond offline classification experiments. The modular separation between ingestion, preprocessing, inference, storage, and visualization makes the pipeline scalable and replaceable: the simulated data generator can be substituted by a live flow collector without changing the downstream inference and monitoring components. This deployment setup therefore supports continuous observation of heterogeneous IoT and IIoT traffic in a practical NIDS integration scenario.

7. Results Discussion

The experimental results show that the proposed ensemble architecture provides robust intrusion classification across heterogeneous IoT and IIoT traffic scenarios. In Use Case 1, the XGBoost meta-learner achieves the strongest overall performance, showing that HTTP-based DDoS attacks benefit from the joint use of spatial graph-based predictions and temporal sequence-based predictions. In Use Case 2, the proposed model remains competitive, although tree-based ensemble baselines obtain slightly better macro-

level scores, suggesting that some application-based DDoS patterns are already well captured by conventional flow-level features. In Use Case 3, the proposed meta-learners achieve the best Macro F1 scores, indicating that spatial-temporal fusion is especially useful when industrial IoT attacks are harder to separate and class imbalance affects model stability.

The main strength of the system is its complementary design. The GAT-XGBoost branch models structural relationships among temporally related flows, while the Residual CNN-BiLSTM branch captures chronological attack behavior through sliding-window sequence modeling. By fusing their predictive distributions, the meta-learner reduces dependence on a single representation type and improves robustness across different traffic conditions. The results also show that the choice of meta-learner can be adapted to the objective: XGBoost is effective for nonlinear fusion, whereas logistic regression can provide stable recall-oriented behavior in more difficult settings.

8. Conclusion

This paper presented an end-to-end ensemble learning architecture for intrusion attack classification. The proposed system combines a GAT embedding XGBoost learner for spatial traffic relationships and a residual CNN-BiLSTM learner for temporal attack patterns. A stacked meta-learner fuses both predictive signals and improves robustness across different IoT and IIoT use cases.

Experiments on IoT-IDS-Zwave-2025 and CIC IIoT 2025 show that the proposed method is competitive with traditional machine-learning baselines, graph-based IDS models, and recent ensemble IDS frameworks. The results confirm that combining spatial and temporal representations is a promising direction for building more reliable NIDS models in heterogeneous traffic environments.

References

- [1] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks (2018). arXiv:1710.10903.
URL <https://arxiv.org/abs/1710.10903>

- [2] T. Chen, C. Guestrin, Xgboost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794. doi:10.1145/2939672.2939785.
- [3] M. Shafi, A. H. Lashkari, Toward generating a large-scale iot-zwave intrusion detection dataset: Smart device profiling, intruders behavior, and traffic characterization, Internet of Things 34 (2025) 101747. doi:10.1016/j.iot.2025.101747.
- [4] A. Firouzi, S. Dadkhah, S. A. Maret, A. A. Ghorbani, Datasense: A real-time sensor-based benchmark dataset for attack analysis in iiot with multi-objective feature selection, Electronics 14 (2025) 4095.
- [5] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, K.-Y. Tung, Intrusion detection system: A comprehensive review, Journal of Network and Computer Applications 36 (1) (2013) 16–24. doi:10.1016/j.jnca.2012.09.004.
- [6] A. L. Buczak, E. Guven, A survey of data mining and machine learning methods for cyber security intrusion detection, IEEE Communications Surveys & Tutorials 18 (2) (2016) 1153–1176. doi:10.1109/COMST.2015.2494502.
- [7] L. Le, Vietnam cybersecurity report 2025, Nhan Dan newspaper (01 2026).
URL <https://nhandan.vn/viet-nam-phai-doi-mat-voi-khoang-552000-cuoc-tan-cong-mang-trong-nam-2025-post936719.html>
- [8] C. Yin, Y. Zhu, J. Fei, X. He, A deep learning approach for intrusion detection using recurrent neural networks, IEEE Access 5 (2017) 21954–21961. doi:10.1109/ACCESS.2017.2762418.
- [9] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, S. Venkatraman, Deep learning approach for intelligent intrusion detection system, IEEE Access 7 (2019) 41525–41550. doi:10.1109/ACCESS.2019.2895334.
- [10] Z. Xu, Y. Wu, S. Wang, J. Gao, T. Qiu, Z. Wang, H. Wan, X. Zhao, Deep learning-based intrusion detection systems: A survey (2025). arXiv:2504.07839.
URL <https://arxiv.org/abs/2504.07839>

- [11] R. Zhao, X. Deng, Z. Yan, J. Ma, Z. Xue, Y. Wang, Mt-flowformer: A semi-supervised flow transformer for encrypted traffic classification, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2022, pp. 2576–2584. doi:10.1145/3534678.3539314.
- [12] A. Naeem, J. Ahmad, M. A. Khan, A. A. Khattak, M. S. Khan, Efficient iot intrusion detection with an improved attention-based cnn-bilstm architecture (2026). arXiv:2503.19339.
URL <https://arxiv.org/abs/2503.19339>
- [13] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: International Conference on Learning Representations, 2017.
URL <https://arxiv.org/abs/1609.02907>
- [14] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems, Vol. 30, 2017.
URL <https://arxiv.org/abs/1706.02216>
- [15] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, M. Portmann, E-graphsage: A graph neural network based intrusion detection system for iot, in: NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium, IEEE, Budapest, Hungary, 2022, pp. 1–9. doi:10.1109/noms54207.2022.9789878.
URL <http://dx.doi.org/10.1109/NOMS54207.2022.9789878>
- [16] K. Hsieh, M. Wong, S. Segarra, S. K. Mani, T. Eberl, A. Panasyuk, R. Netravali, R. Chandra, S. Kandula, NetVigil: Robust and Low-Cost anomaly detection for East-West data center security, in: 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), USENIX Association, Santa Clara, CA, 2024, pp. 1771–1789.
URL <https://www.usenix.org/conference/nsdi24/presentation/hsieh>
- [17] W. Villegas, J. Govea, A. Navarro, P. Palacios, Intrusion detection in iot networks using dynamic graph modeling and graph-based neural networks, IEEE Access PP (2025) 1–1. doi:10.1109/ACCESS.2025.3559325.

- [18] M. Adil, M. A. Jan, S. B. Hakim, H. H. Song, Z. Jin, xids-ensembleguard: An explainable ensemble learning-based intrusion detection system (2025). arXiv:2503.00615.
URL <https://arxiv.org/abs/2503.00615>
- [19] S. Nugroho, A stacking ensemble learning framework for network intrusion detection using xgboost, lightgbm, catboost, and adaboost with random forest meta model, in: 2026 International Conference on Communication Networks and Machine Learning (CNML), 2026, pp. 20–23. doi:10.1109/CNML68938.2026.11452388.
- [20] B. Ahmad, Z. Wu, Y. Huang, S. U. Rehman, Enhancing the security in iot and iiot networks: An intrusion detection scheme leveraging deep transfer learning, Knowledge-Based Systems 305 (2024) 112614. doi:10.1016/j.knosys.2024.112614.
- [21] F. Nie, W. Liu, G. Liu, B. Gao, M2VT-IDS: A multi-task multi-view learning architecture for designing iot intrusion detection system, Internet of Things 25 (2024) 101102. doi:10.1016/j.iot.2024.101102.
- [22] S. Hizal, U. Cavusoglu, D. Akgun, A novel deep learning-based intrusion detection system for iot ddos security, Internet of Things 28 (2024) 101336. doi:10.1016/j.iot.2024.101336.
- [23] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778. doi:10.1109/CVPR.2016.90.
- [24] D. Bahdanau, K. Cho, Y. Bengio, Neural machine translation by jointly learning to align and translate, in: International Conference on Learning Representations, 2015.
URL <https://arxiv.org/abs/1409.0473>
- [25] J. Hu, L. Shen, S. Albanie, G. Sun, E. Wu, Squeeze-and-excitation networks (2019). arXiv:1709.01507.
URL <https://arxiv.org/abs/1709.01507>
- [26] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, S. Y. Philip, A comprehensive survey on graph neural networks, IEEE Transactions on Neural Networks and Learning Systems 32 (1) (2021) 4–24. doi:10.1109/TNNLS.2020.2978386.

- [27] M. Termos, Z. Ghalmane, M.-E.-A. Brahmia, A. Fadlallah, A. Jaber, M. Zghal, GDLC: A new graph deep learning framework based on centrality measures for intrusion detection in iot networks, *Internet of Things* 26 (2024) 101214. doi:10.1016/j.iot.2024.101214.
- [28] T. K. Rusch, M. M. Bronstein, S. Mishra, A survey on oversmoothing in graph neural networks (2023). arXiv:2303.10993.
URL <https://arxiv.org/abs/2303.10993>
- [29] Z. Goldfeld, Y. Polyanskiy, The information bottleneck problem and its applications in machine learning (2020). arXiv:2004.14941.
URL <https://arxiv.org/abs/2004.14941>
- [30] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, Lightgbm: A highly efficient gradient boosting decision tree, in: *Advances in Neural Information Processing Systems*, Vol. 30, 2017.
- [31] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, A. Gulin, Catboost: Unbiased boosting with categorical features, in: *Advances in Neural Information Processing Systems*, Vol. 31, 2018.
- [32] J. Zhu, X. Liu, An integrated intrusion detection framework based on subspace clustering and ensemble learning, *Computers and Electrical Engineering* 115 (2024) 109113. doi:10.1016/j.compeleceng.2024.109113.
- [33] P. R. C. Rani, K. Baalaji, Deep learning-based ensemble stacking for enhanced intrusion detection in iot-edge platforms, *Discover Applied Sciences* 7 (8) (2025) 928. doi:10.1007/s42452-025-06871-z.
- [34] S. Brody, U. Alon, E. Yahav, How attentive are graph attention networks? (2022). arXiv:2105.14491.
URL <https://arxiv.org/abs/2105.14491>
- [35] N. Moustafa, J. Slay, Unsw-nb15: A comprehensive data set for network intrusion detection systems, *Military Communications and Information Systems Conference* (2015) 1–6doi:10.1109/MilCIS.2015.7348942.
- [36] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: *Proceedings of the 4th International Conference*

- on Information Systems Security and Privacy, 2018, pp. 108–116. doi:10.5220/0006639801080116.
- [37] N. Koroniotis, N. Moustafa, E. Sitnikova, B. Turnbull, Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset, *Future Generation Computer Systems* 100 (2019) 779–796. doi:10.1016/j.future.2019.05.041.
- [38] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, A. Anwar, Ton_iot telemetry dataset: A new generation dataset of iot and iiot for data-driven intrusion detection systems, *IEEE Access* 8 (2020) 165130–165150. doi:10.1109/ACCESS.2020.3022862.